

Programming Fundamentals

COMPLETE 50 QUESTIONS & ANSWERS GUIDE

Question 1: What is a variable in C/C++?

A variable is a named storage location in the computer's memory (RAM) used to hold data that can be modified during program execution. It acts as a container associated with an identifier (name), data type, and a specific memory address.

Question 2: Why do we use variables in programs?

Variables are used to dynamically store, retrieve, and manipulate data throughout a program. Without variables, data values would be hardcoded, making it impossible to process dynamic user inputs, track changes, or reuse values efficiently.

Question 3: What is the difference between a variable name and its value?

The **variable name** (identifier) is the label used in the source code to refer to the memory location. The **value** is the actual content stored inside that memory location. For example, in `int x = 5;`, `x` is the name, and `5` is the value.

Question 4: Which of the following is a valid variable name?

a) 2num b) total_marks c) for d) my marks

The correct answer is **b) total_marks**.

- 2num is invalid because variable names cannot start with a digit.
- for is invalid because it is a reserved keyword in C/C++.
- my marks is invalid because spaces are not permitted in variable names.

Question 5: Why is Age different from age?

C/C++ is a **case-sensitive** programming language. The compiler treats uppercase and lowercase letters as completely distinct characters. Therefore, `Age` and `age` refer to two completely separate memory locations.

Question 6: Can a variable name contain spaces? Why?

No, a variable name cannot contain spaces. The compiler interprets a space as a separator between distinct tokens or identifiers. A space in a name would break the syntax and result in a compilation error.

Question 7: Can a variable name start with an underscore (_) ?

Yes, a variable name can legally start with an underscore (`_`). However, it is generally discouraged for general application programming because names starting with an underscore followed by an uppercase letter or another underscore are reserved for system identifiers and standard library internal implementations.

Question 8: What happens if two variables have the same name in the same scope?

It will result in a **compilation error** (redefinition/redeclaration error). The compiler cannot uniquely identify which memory location is being referred to when that name is called.

Question 9: Write three meaningful variable names for storing student information.

Three meaningful and descriptive variable names using camelCase are:

- `studentRollNumber`
- `studentGpa`
- `studentEmailAddress`

Question 10: Which naming convention is better and why: `studentMarks` or `sm`?

`studentMarks` is much better. It is descriptive and explicitly explains what data the variable contains, making the code highly readable and self-documenting. The abbreviation `sm` is ambiguous and makes code maintenance difficult.

Question 11: What is a data type?

A data type is a classification attribute that tells the compiler what type of value a variable can store, how much memory space to allocate for it (e.g., 4 bytes for an `int`), and what operations can be validly performed on that data.

Question 12: Which data type is used to store whole numbers?

The `int` (integer) data type is used to store positive or negative whole numbers without any fractional or decimal component.

Question 13: Which data type is used to store decimal values?

The `float` or `double` data types are used to store numbers containing a fractional or decimal component.

Question 14: What is the difference between `float` and `double`?

The primary differences lie in size and precision:

- `float` typically occupies **4 bytes** of memory and provides about 7 digits of decimal precision.
- `double` occupies **8 bytes** of memory and provides about 15 digits of precision, which is twice the precision of a `float`.

Question 15: Which data type stores a single character?

The `char` data type is used to store a single character, enclosed in single quotes (e.g., `'A'`). It typically occupies 1 byte of memory.

Question 16: Which data type stores True/False values?

The `bool` (boolean) data type is used to store logical values, which can only be either `true` or `false`.

Question 17: What data type is suitable for storing a person's name?

The `std::string` class data type (available by including the `<string>` header) is suitable, as it stores a sequence of characters.

Question 18: Predict the output: `int age=18; cout<<age;`

The output is:

```
18
```

Question 19: Identify the data type: 25, 3.14, 'A', true.

The standard data types are:

- 25 : `int`
- 3.14 : `double` (by default in C++) or `float`
- 'A' : `char`
- true : `bool`

Question 20: What happens if you store a decimal number in an int variable?

The compiler will perform **implicit truncation**. The fractional/decimal part will be discarded entirely without rounding up. For example, storing 5.99 into an `int` variable results in the value becoming 5.

Question 21: What is the purpose of `cin`?

`cin` stands for "character input". It is a standard input stream object in C++ (defined in `<iostream>`) used to read data from the standard input device, which is usually the keyboard.

Question 22: What is the purpose of `cout`?

`cout` stands for "character output". It is a standard output stream object in C++ used to print or display data to the standard output device, which is typically the monitor screen.

Question 23: Which symbol is used with `cin`?

The **extraction operator** or right shift symbol `>>` is used with `cin`.

Question 24: Which symbol is used with `cout`?

The **insertion operator** or left shift symbol `<<` is used with `cout`.

Question 25: Explain: `cin >> age;`

This statement pauses program execution to wait for user input from the keyboard. When the user types a value and presses Enter, the input stream extracts that value and saves it into the variable named `age`.

Question 26: Explain: `cout << age;`

This statement takes the value currently stored inside the variable `age` and sends/inserts it into the standard output stream, displaying it on the console screen.

Question 27: What is the purpose of `endl`?

`endl` stands for "end line". It performs two actions: it inserts a newline character (`' \n '`) into the output stream moving the cursor to the next line, and it immediately flushes the output stream buffer to ensure all text is printed.

Question 28: Write a statement to input a student's marks.

```
cin >> studentMarks;
```

Question 29: Write a statement to display 'Welcome to C++'.

```
cout << "Welcome to C++";
```

Question 30: Predict the output: `cout << 'Programming';`

This statement will cause a **compilation error** or unexpected behavior because single quotes are reserved for single characters only (a multi-character character constant overflow). To print a text string, double quotes must be used: `cout << "Programming";`.

Question 31: What is an operator?

An operator is a special symbol that tells the compiler to perform a specific mathematical, logical, or relational manipulation on data elements called operands.

Question 32: Which operator is used for addition?

The plus symbol `+` is used for addition.

Question 33: What is the result of `10 % 3`?

The answer is **1**. The modulo operator (`%`) returns the remainder of integer division. When 10 is divided by 3, the quotient is 3 and the remainder is 1.

Question 34: What is the result of `8 % 2`?

The answer is **0**. Because 8 is completely divisible by 2, there is no remainder left.

Question 35: Which operator checks equality?

The double equal-to sign `==` is the equality relational operator.

Question 36: What is the difference between = and == ?

- = is the **assignment operator**. It copies the value from its right-hand side into the variable on its left-hand side (e.g., `x = 5;`).
- == is the **equality comparison operator**. It evaluates whether the values on both sides are equal and returns a boolean value (true or false).

Question 37: What is the result of `5 > 3`?

The result is `true` (which is evaluated numerically as the integer 1 in C++ output streams).

Question 38: What does `&&` represent?

It represents the **Logical AND** operator. It combines two expressions and returns `true` only if both component conditions evaluate to true.

Question 39: What does `||` represent?

It represents the **Logical OR** operator. It combines two conditions and returns `true` if at least one of the conditions evaluates to true.

Question 40: What does `!` represent?

It represents the **Logical NOT** (unary negation) operator. It reverses the logical state of an operand, transforming `true` to `false` and vice versa.

Question 41: Write a program to print 'Hello World'.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World" << endl;
    return 0;
}
```

Question 42: Write a program to input two numbers and display their sum.

```
#include <iostream>
using namespace std;

int main() {
    double num1, num2, sum;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    sum = num1 + num2;
    cout << "The sum is: " << sum << endl;
    return 0;
}
```

Question 43: Write a program to calculate the area of a rectangle.

```
#include <iostream>
using namespace std;

int main() {
    double length, width, area;
    cout << "Enter length of rectangle: ";
    cin >> length;
    cout << "Enter width of rectangle: ";
    cin >> width;

    area = length * width;
    cout << "Area of the rectangle is: " << area << endl;
    return 0;
}
```

Question 44: What formula is used to calculate Simple Interest?

The formula used is:

$$SI = (P \times R \times T) / 100$$

Where **P** = Principal amount, **R** = Rate of interest per annum, and **T** = Time duration in years.

Question 45: Write a program to calculate Simple Interest.

```
#include <iostream>
using namespace std;

int main() {
    double principal, rate, time, simpleInterest;
    cout << "Enter Principal amount: ";
    cin >> principal;
    cout << "Enter Annual Interest Rate (%): ";
    cin >> rate;
    cout << "Enter Time period (in years): ";
    cin >> time;

    simpleInterest = (principal * rate * time) / 100.0;
    cout << "Calculated Simple Interest is: " << simpleInterest << endl;
    return 0;
}
```

Question 46: Why is a temporary variable used while swapping two numbers?

A temporary variable is used as a holding container. If you assign the value of the second variable directly to the first variable ($a = b$), the original value stored in the first variable (a) gets overwritten and permanently lost. The temporary variable preserves this original value so it can be assigned back to b .

Question 47: Swap the values: $a=10$, $b=20$.

Using a temporary variable workflow:

```
int a = 10, b = 20;
int temp;

temp = a; // temp gets 10
a = b;    // a gets 20
b = temp; // b gets 10
```

Question 48: What formula is used to convert Celsius into Fahrenheit?

The mathematical conversion formula is:

$$F = (C \times 9/5) + 32$$

Or expressed in decimal notation: $F = (C \times 1.8) + 32$.

Question 49: What is the Fahrenheit value of 0°C ?

Substituting $C = 0$ into the formula: $F = (0 \times 1.8) + 32 = 32$. Therefore, 0°C evaluates to 32°F .

Question 50: Why can integer division give incorrect results in the Celsius-to-Fahrenheit program?

In C/C++, evaluating the fraction expression $9 / 5$ using integer data types triggers integer division, discarding the fractional parts and resulting in 1 instead of 1.8. To evaluate the formula accurately, floating-point notation must be specified using double elements, like $9.0 / 5.0$ or 1.8 .